
Uso de `@riverpod` con anotaciones

1. Configuración inicial

En tu `pubspec.yaml` asegúrate de incluir:

```
dependencies:  
  flutter_riverpod: ^3.3.0  
  riverpod_annotation: ^3.3.0  
  
dev_dependencies:  
  build_runner: ^2.4.0  
  riverpod_generator: ^3.3.0
```

Ejecuta:

```
flutter pub get
```

2. Providers con `@riverpod`

a) Provider simple (valor inmutable)

Se usa para exponer valores constantes o configuraciones.

```
import 'package:riverpod_annotation/riverpod_annotation.dart';  
part 'config.g.dart';  
  
@riverpod  
String appName(AppNameRef ref) => 'MiAplicación';
```

Uso en UI:

```
final name = ref.watch(appNameProvider);
```

b) State Provider (estado simple y mutable)

Ideal para valores que cambian, como un contador o un booleano.

```
@riverpod
class Counter extends _$Counter {
  @override
  int build() => 0;

  void increment() => state++;
  void decrement() => state--;
}
```

Uso en UI:

```
final count = ref.watch(counterProvider);
ref.read(counterProvider.notifier).increment();
```

c) Future Provider (estado asincrónico)

Para llamadas HTTP o procesos que devuelven un `Future`.

```
import 'package:http/http.dart' as http;
import 'dart:convert';

@riverpod
Future<List<String>> posts(PostsRef ref) async {
  final response = await
  http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts'));
  final List data = jsonDecode(response.body);
  return data.map((e) => e['title'] as String).toList();
}
```

Uso en UI:

```
final postsAsync = ref.watch(postsProvider);
```

d) Stream Provider (flujos continuos)

Para datos que llegan en tiempo real, como sockets o relojes.

```
@riverpod
Stream<DateTime> clock(ClockRef ref) async* {
  while (true) {
    await Future.delayed(Duration(seconds: 1));
    yield DateTime.now();
  }
}
```

```
}  
}
```

Uso en UI:

```
final timeAsync = ref.watch(clockProvider);
```

e) Notifier avanzado (lógica de negocio compleja)

Cuando necesitas manejar listas, mutaciones o lógica más elaborada.

```
@riverpod  
class TodoList extends _$TodoList {  
  @override  
  List<String> build() => [];  
  
  void add(String todo) => state = [...state, todo];  
  void remove(String todo) => state = state.where((t) => t != todo).toList();  
}
```

Uso en UI:

```
final todos = ref.watch(todoListProvider);  
ref.read(todoListProvider.notifier).add('Nueva tarea');
```

3. Generación automática

Cada vez que declares un provider con `@riverpod`, ejecuta:

```
flutter pub run build_runner build
```

Esto generará los archivos `.g.dart` con los providers listos para usar.

✓ Conclusión

- `@riverpod` permite definir **todos los tipos de provider** (simples, estado, futuros, streams, notifiers) con menos código repetitivo.
- La lógica se mantiene clara y escalable.
- La generación automática asegura que los nombres de providers estén disponibles sin necesidad de escribirlos manualmente.

